
AUTOMATED URL DISCOVERY AND SEMANTIC RELEVANCE FILTER – AUDSRF

Prepared by: Olanrewaju Adegoke

Submitted to: WHITE LIGHT DIGITAL MARKETING (WLDM)

GitHub Repository (Private): <https://github.com/larrysman/automated-url-discovery-and-semantic-relevance-filter/tree/main>
(Available on request)

EXECUTIVE SUMMARY

This project delivers an end-to-end automated system capable of transforming a noisy list of root domains into a clean, high-precision dataset of niche relevant deep URLs. The solution addresses a common real-world challenge, organizations often possess domain-level data but lack visibility into the specific content pages that matter for research, SEO, competitive intelligence, or content acquisition.

The digital marketing teams often need to analyze large sets of domains to identify pages that align with specific campaign keywords and manually performing this research is slow, inconsistent, and difficult to scale.

To solve this, an 8-stage pipeline was developed using Python, combining structured crawling, metadata extraction, semantic relevance scoring, and automated filtering. The system ingests two CSV files, one containing raw domains and another containing target-specific keywords and expands each domain to discover deeper content pages. These pages are then evaluated using lexical and semantic similarity techniques to determine relevance.

The final output is a clean spreadsheet containing only high-value URLs, each annotated with its source domain, detected keyword, and relevance score. The pipeline is noise-tolerant, scalable, and designed for reproducibility, making it suitable for production-level workflows or integration into larger content intelligence systems and the 8-stage pipeline include:

- Load and Normalizes input domains and target keywords data
- Crawls multiple domains
- Discover URLs
- Extract contents
- Evaluates semantic relevance
- Prioritizes pages
- Outputs a clean CSV for decision-making.

The submission herewith will include:

- Full source code – exactly as implemented
 - Final CSV output snapshot generated from the sample dataset
 - Technical documentation
 - Architectural Flows and Diagrams
 - Performance notes
 - Limitations and future improvements.
-

INTRODUCTION

This report documents the design, implementation, and evaluation of an automated pipeline developed to identify high-quality, keyword relevant deep URLs from a noisy list of root domains. The system integrates web crawling, metadata extraction, semantic relevance scoring, and structured output generation into a reproducible, modular workflow.

The solution was built entirely in Python and follows an 8-stage pipeline architecture designed for robustness, extensibility, and real-world data noise tolerance.

PROJECT OVERVIEW

Modern content discovery tasks often require navigating large volumes of unstructured or low-quality domain data. The challenge addressed in this project is to:

- Start from a CSV containing **root domains** only e.g *stadinraksat.fi*

- Automatically crawl each domain to discover deep pages e.g, *stadinraksat.fi/topic/article*
- Filter these pages using a semantic relevance engine based on a second CSV containing target keywords
- Produce a clean, high-precision spreadsheet of only the URLs that match the target keywords.

The pipeline is designed to handle messy, real-world data containing irrelevant, low-quality, or off-topic domains.

PROBLEM STATEMENT

The input dataset contains only homepages, not the actual content pages needed for analysis. The core problem is therefore in two fold:

1. URL Expansion:

Automatically explore each domain to uncover deeper content pages such as articles, guides, reviews, and topic-specific resources.

2. Semantic Filtering:

Evaluate each discovered page against a curated list of target keywords and determine whether it is relevant.

The system must also exclude:

Utility pages (About, Contact, Privacy Policy), Generic news feeds, Off-topic or low-value content.

INPUT DATA DESCRIPTION

input_domains.csv:

- Contains raw root domains
- Includes high-value and irrelevant junk domains that requires cleaning
- Requires normalization and validation before crawling.

```
> input_domains.csv > data
Domain
stadinraksat.fi
rodokselle.fi
autotjaliikenne.fi
vinkkaajaveikkaa.fi
casinopelitnetissa.com
companionlink.com/blog
```

target_keyword.csv:

- Contains target specific keywords
- Defines the semantic boundaries of relevance
- Used for lexical and semantic scoring.

```
> target_keywords.csv > data
Topic
Lucky Neko
Mahjong Ways
Chilli Heat Megaways
Extra Juicy Megaways
Floating Dragon Hold Spin Megaways
Madame Destiny Megaways
```

END-TO-END PIPELINE ARCHITECTURE (8 STAGES)

The solution is implemented as a modular pipeline with eight clearly defined stages.

Stage 01 : Load, Clean and Normalize Data

Load both CSV files, clean domain formats, remove duplicates, and validate keyword entries.

Key Operations include:

- Strip protocols (http://, https://)
- Remove trailing slashes
- Remove duplicate (deduplicate)
- Validate keyword list.
- Run diagnostics of the input data

Script: *load_and_clean_data_01.py*

```
STAGE 01 - LOAD AND NORMALIZE WITH DIAGNOSTICS SUMMARY:  
{'initial_domain_count': 11, 'initial_keyword_count': 961, 'removed_invalid_domains': 0, 'removed_empty_keywords': 0, 'duplicates_removed_domains': 0, 'duplicates_removed_keywords': 0}
```

Stage 02: Domain Pre-processing

Prepare each domain for crawling by applying lightweight heuristics and the purpose is to ensure that higher priority domains are crawled first thereby improving efficiency.

Operations:

- Check domain reachability
- Detect homepage text length
- Identify potential contents management system patterns
- Compute keyword density on homepage
- Assign domain priority score.

Script: *domain_preprocessing_02.py*

Stage 03 : Site Expansion and Crawling Initialization

Initialize a structured queue of domains and URLs to crawl with configuration parameters for better performance. The parameters include: MAX_DEPTH, MAX_PAGES_PER_DOMAIN, SAME_DOMAIN_ONLY, POLITE_DELAY, TIMEOUT.

Script: *site_exp_and_crawling_initiate_03.py*

Stage 04 : Crawling and Deep URL Discovery

This expand the domain to discover internal URLs and follows the logic by starting from the homepage, extract internal links, normalize and remove duplicate URLs, prioritize URLs containing keyword-like slugs to avoiding infinite loops and external domains.

The outputs expected are: list of candidate deep URLs per domain and the metadata extracted such as the Title tag, H1, URL slug.

Script: *crawling_and_deep_url_discovery_04.py*

Stage 05 : Semantic Relevance Filtering

Score each discovered page using lexical and semantic similarity and the components include:

- **Lexical Match Score:** This is the direct keyword matches in Title, H1, URL slug and body text.
- **Semantic Similarity Score:** This uses the TF_IDF vectorization and cosine similarity for the case of this project. More sophisticated architecture may also be used such as the BM25 or FAISS etc.
- **Utility Page Exclusion:** Automatic removal of the /about, /contact, /privacy and the /terms pages.

This output list of pages with relevance scores and detected keywords.

Script: *semantic_relevance_filtering_05.py*

Stage 06: Assembling Output – CSV Formatting

Converted filtered results into a clean, structured DataFrame with required columns and handles if the dataframe is empty. The required columns are:

| Source Domain | Target URL | Detected Keyword | Relevance Score |

Operations

- Validate the semantic results
- Rank URLs by score
- Format into final DataFrame
- Handles if DataFrame is empty.

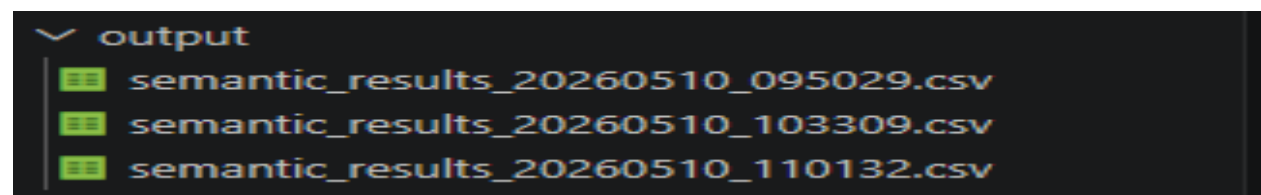
Script: *assembling_output_06.py*

Stage 07 : Final CSV Output Generation

Save the final generated dataframe into a CSV file with timestamp for versioning in the /output directory of the project root.

Output File:

output/semantic_results_YearMonthDay_HourMinuteSecond.csv



Script:
final_csv_output_generator_07.py

on_07.py

Stage 08 : Quality Checks and Success Metrics

This evaluate the performance of the pipeline by measuring the metrics for precision, granularity and suspicious pages.

Metrics:

- Precision: This measures the percentage of discovered URLs that are relevant.
- Granularity: This measures the ratio of deep URLs against the homepage.
- Relevance distribution
- Suspicious Pages: This flags a low score or off topic pages for review.

Implementation includes:

- Deep URL ration
- Relevance score distribution
- Domain level match counts

Output:

```
precision: 1.0
granularity: {'addis-techblog.de': 3, 'internetblogger.de': 2, 'autotjaliikenne.fi': 2, 'companionlink.com': 1, 'deutsche-staedte.de': 1}
relevance_distribution: {'min_score': 0.16, 'max_score': 0.16, 'mean_score': 0.16, 'median_score': 0.16}
potential_off_topic_pages: ['https://addis-techblog.de/it/smarthome', 'https://addis-techblog.de/it/hardware', 'https://addis-techblog.de/it/software', 'https://internetblogger.de/2026/05/jumia-technologies-aktie-wkn-a2pgzm-warum-die-aktie-nach-dem-kurssprung-wieder-gefallen-ist', 'https://internetblogger.de/2026/05/ebay-aktualisiert-die-zahlungsbedingungen-das-musst-du-wissen', 'https://autotjaliikenne.fi/tahtia-utot-esittelyssa-unohdumattomimmat-tvsta-tutut-autot', 'https://autotjaliikenne.fi/live-vedonlyonti-ja-moottoriurheilu', 'https://companionlink.com/goldmine', 'https://deutsche-staedte.de/bergstrasse/weinbaue-biet.php?start=5']
```

Complete Pipeline output running from the terminal:

```
STAGE 01 - LOAD AND NORMALIZE WITH DIAGNOSTICS SUMMARY:
{'initial_domain_count': 11, 'initial_keyword_count': 961, 'removed_invalid_domains': 0, 'removed_empty_keywords': 0, 'duplicates_removed_domains': 0, 'duplicates_removed_keywords': 0}
STAGE 02 - PREPROCESSING THE DOMAINS: 11
STAGE 03 - INITIAL QUEUE SIZE: 10
STAGE 04 - TOTAL URLS DISCOVERED: 1674
STAGE 05 - RELEVANT URLS AFTER SEMANTIC FILTERING: 9
STAGE 06 - FINAL CSV AND SPREADSHEET ROWS: 9
STAGE 07 - CSV exported to: output/semantic_results_20260510_110132.csv
STAGE 08 - QUALITY REPORT:
```

DELIVERABLES

Source Code

All Python scripts (ONLY ORCHESTRATOR) implementing the 8-stage pipeline:

STAGE 01:

```
# LOAD THE INPUT DOMAINS DATA AND TARGET KEYWORD DATA

import numpy as np
import pandas as pd
import os
from typing import Tuple, Dict
import re

def load_and_normalize_data(
    input_domains_path: str,
    target_keywords_path: str
) -> Tuple[pd.DataFrame, pd.DataFrame, Dict]:
    """
    THIS FUNCTION LOADS, VALIDATES, CLEANS, AND NORMALIZES THE INPUT DOMAINS AND TARGET KEYWORDS.

    Args:
        input_domains_path | str: PATH TO THE INPUT DOMAINS CSV FILE.
        target_keywords_path | str: PATH TO THE TARGET KEYWORDS CSV FILE.

    Returns:
        cleaned_domains_df | pd.DataFrame: CLEANED AND NORMALIZED DOMAINS
        cleaned_keywords_df | pd.DataFrame: CLEANED KEYWORDS.
        diagnostics | Dict: SUMMARY OF ISSUES FOUND DURING CLEANING
    """
    # -----
    # LOADING THE INPUT RAW DATA - DOMAINS AND KEYWORDS
    # -----
    domains_df = pd.read_csv(input_domains_path)
    keywords_df = pd.read_csv(target_keywords_path)

    diagnostics_check = {
        "initial_domain_count": len(domains_df),
        "initial_keyword_count": len(keywords_df),
        "removed_invalid_domains": 0,
        "removed_empty_keywords": 0,
        "duplicates_removed_domains": 0,
        "duplicates_removed_keywords": 0
    }

    # -----
    # VALIDATING REQUIRED COLUMNS FOR THE INPUT RAW DATA - DOMAINS AND KEYWORDS
    # -----
    if domains_df.shape[1] != 1:
        raise ValueError("input domains data must contain exactly ONE column.")

    if keywords_df.shape[1] != 1:
        raise ValueError("input keywords data must contain exactly ONE column.")

    # RENAMING THE DATASETS COLUMNS
    domains_df.columns = ["domain"]
    keywords_df.columns = ["keyword"]

    # -----
    # CLEAN AND NORMALIZE THE DOMAINS CSV
    # -----
    def normalize_domains(df: pd.DataFrame) -> pd.DataFrame:
        if not isinstance(df, pd.DataFrame):
            return None

        domain = df["domain"].str.lower()

        # REMOVE EXISTING PROTOCOLS IN THE DOMAIN
        domain = re.sub(r"(https?://)", "", domain)

        # REMOVE EXISTING WWW. IN THE DOMAIN
        domain = re.sub(r"(www\.)", "", domain)

        # REMOVE PATHS AND QUERY PARAMETERS FROM DOMAIN
        domain = domain.split("/")[0]
        domain = domain.split("?")[0]

        # REMOVING TRAILING DOTS OR SLASHES FROM DOMAIN
        domain = domain.rstrip("/").rstrip(".")

        return domain if domain else None

    # APPLIED THE NORMALIZED DOMAINS DF TO THE DOMAINS DF AND CREATED A NEW COLUMN CALLED NORMALIZED
    domains_df["normalized"] = domains_df["domain"].apply(normalize_domains)
```

```

# DETECT MALFORMED DOMAINS
def _is_valid_domain(domain: str) -> bool:
    if not isinstance(domain, str):
        return False
    if "*" in domain:
        return False
    if "." not in domain:
        return False
    return True

domains_df["valid"] = domains_df["normalized"].apply(_is_valid_domain)

diagnostics_check["removed_invalid_domains"] = len(domains_df[~domains_df["valid"]])

# KEEP ONLY VALID DOMAINS
cleaned_domains_df = domains_df[domains_df["valid"]].copy()

# REMOVING DUPLICATES
before = len(cleaned_domains_df)
cleaned_domains_df = cleaned_domains_df.drop_duplicates(subset=["normalized"])
diagnostics_check["duplicates_removed_domains"] = before - len(cleaned_domains_df)

cleaned_domains_df = cleaned_domains_df[["normalized"]].rename(columns={"normalized": "domain"})

# -----
# CLEAN AND NORMALIZE KEYWORDS
# -----
def _clean_keywords(keyword: str) -> str:
    if not isinstance(keyword, str):
        return None
    keyword = keyword.strip().lower()
    return keyword if keyword else None

keywords_df["cleaned"] = keywords_df["keyword"].apply(_clean_keywords)

# REMOVE EMPTY OR INVALID KEYWORDS
diagnostics_check["removed_empty_keywords"] = len(keywords_df[keywords_df["cleaned"].isna()])

cleaned_keywords_df = keywords_df[keywords_df["cleaned"].notna()].copy()

# REMOVING DUPLICATES FOR THE KEYWORD DATASET
before = len(cleaned_keywords_df)
cleaned_keywords_df = cleaned_keywords_df.drop_duplicates(subset=["cleaned"])
diagnostics_check["duplicates_removed_keywords"] = before - len(cleaned_keywords_df)

cleaned_keywords_df = cleaned_keywords_df[["cleaned"]].rename(columns={"cleaned": "keyword"})

# -----
# RETURN CLEANED DATA AND THE SUMMARIZED DIAGNOSTICS
# -----
return cleaned_domains_df.reset_index(drop=True), cleaned_keywords_df.reset_index(drop=True), diagnostics_check

```

STAGE 02:

```
# STAGE 02 - DOMAIN PRE-PROCESSING - DEEP EXPLORATION
"""
TAKES CLEANED INPUT FROM STAGE 01 AND PREPARES IT FOR CRAWLING
TURN A LIST OF NORMALIZED DOMAINS INTO STRUCTURED, VALIDATED, CRAWL-READY DATAFRAME.

THIS STAGE IS THE BRIDGE BETWEEN RAW CSV INOUT AND A CRAWLER-READY DOMAIN LIST.
"""

import requests
from bs4 import BeautifulSoup
from typing import Tuple
import pandas as pd

def preprocessing_domains(domains_df: pd.DataFrame) -> pd.DataFrame:
    """
    PERFORMS DOMAIN PRE-PROCESSING TO PREPARE FOR CRAWLING.
    INCLUDES CANONICALIZATION (THE FINAL NORMALIZATION PASS), HEALTH CHECKS (CRITICAL FILTERING),
    AND HOMEPAGE FETCH AND METADATA EXTRACTION.

    Args:
    | domains_df | pd.DataFrame: CLEANED DOMAINS FROM STAGE 1.

    Returns:
    | pd.DataFrame: CRAWL-READY DOMAIN DATAFRAME WITH METADATA.
    """

    def canonicalize_for_final_normalization(domain: str) -> str:
        domain = domain.strip().lower()
        return domain

    # FETCH THE HOMEPAGE BASED ON THE INPUT DOMAIN
    def fetch_the_homepage(domain: str):
        url = f"https://{domain}"
        try:
            response = requests.get(url, timeout=8, headers={"User-Agent": "Mozilla/5.0"})
            if response.status_code >= 400:
                return None, None, None, None, "bad_status"
            return url, response.text, response.status_code, response.headers, "ok"
        except Exception:
            return None, None, None, None, "error"

    # EXTRACT TAGS FOR TITLE, H1 AND INTERNAL LINKS
    def extract_metadata_from_homepage(html: str):
        if not html:
            return None, None, 0

        soup = BeautifulSoup(html, "html.parser")

        # THE SEO TITLE OF THE PAGE
        # title = soup.title.string.strip() if soup.title else None
        title = soup.title.get_text(strip=True) if soup.title else None

        # THE MAIN HEADING OF THE PAGE
        h1_tag = soup.find("h1")
        h1 = h1_tag.get_text(strip=True) if h1_tag else None

        # ITERATE OVER THE PAGE FOR HYPERTEXT REFERENCE (HREF)
        internal_links = len([
            tag["href"] for tag in soup.find_all("a", href=True)
            if tag["href"].startswith("/") or tag["href"].startswith("#")
        ])

        return title, h1, internal_links
```

```

# PREPARE THE OUTPUT STRUCTURE FROM THE EXTRACTED METADATA FROM THE PAGE
results = []

for _, row in domains_df.iterrows():
    domain = canonicalize_for_final_normalization(row["domain"])

    homepage_url, html, status_code, headers, status = fetch_the_homepage(domain)

    if status != "ok":
        results.append({
            "domain": domain,
            "homepage_url": homepage_url,
            "status": status,
            "homepage_title": None,
            "homepage_h1": None,
            "internal_links": 0,
            "is_valid": False
        })
        continue

    title, h1, internal_links = extract_metadata_from_homepage(html)

    results.append({
        "domain": domain,
        "homepage_url": homepage_url,
        "status": "ok",
        "homepage_title": title,
        "homepage_h1": h1,
        "internal_links": internal_links,
        "is_valid": True
    })

return pd.DataFrame(results)

```

STAGE 03:

```

# ORCHESTRATOR SCRIPTS FOR INITIALIZING THE CRAWL QUEUE
# -----
def initialize_crawl_queue(domains_df: pd.DataFrame, keywords_df: pd.DataFrame) -> pd.DataFrame:
    """
    BUILDS THE INITIAL CRAWL QUEUE USING THE CONFIGURATION SETTINGS FROM THE PREPROCESSED DOMAINS.
    EACH DOMAIN CONTRIBUTES ITS HOMEPAGE AS A SEED URL.
    """

    keywords = keywords_df["keyword"].tolist()

    queue = []
    already_seen = initialize_seen_set()

    for _, row in domains_df.iterrows():
        domain = row["domain"]
        homepage_url = row.get("homepage_url", build_homepage_url(domain))

        # COMPUTE PRIORITY USING CONFIG WEIGHTS
        priority = compute_priority_score(
            homepage_title=row.get("homepage_title"),
            homepage_h1=row.get("homepage_h1"),
            internal_links=row.get("internal_links", 0),
            keywords=keywords
        )

        # CREATE QUEUE ENTRY
        entry = create_queue_entry(
            url=homepage_url,
            domain=domain,
            depth=0,
            priority=priority,
            source="seed"
        )

        # ADD TO QUEUE WITH DUPLICATE PROTECTION
        add_to_queue(queue, already_seen, entry)

    # CONVERT TO DATAFRAME
    queue_df = pd.DataFrame(queue)

    # ATTACH CONFIGURATION VALUES FOR DOWNSTREAM STAGES
    queue_df["max_depth"] = MAX_CRAWL_DEPTH
    queue_df["max_pages_per_domain"] = MAX_PAGES_PER_DOMAIN

    return queue_df

```


STAGE 04:

```
# -----
# ORCHESTRATOR SCRIPTS FOR THE CRAWL - INTEGRATED ALL MODULES IN STAGE 4 TOGETHER
# -----
def orchestrate_crawl_query(queue_df: pd.DataFrame) -> pd.DataFrame:
    """
    MAIN CRAWLING LOOP: FETCHES PAGES, EXTRACTS LINKS, EXPANDS QUEUE AND RETURNS AN UPDATED QUEUE WITH DISCOVERED URLS.
    """

    queue = queue_df.to_dict("records")
    already_seen = set([entry["url"] for entry in queue])
    domain_page_count = {}

    i = 0
    while i < len(queue):
        entry = queue[i]
        domain = entry["domain"]

        # INITIALIZE DOMAIN COUNT
        domain_page_count.setdefault(domain, 0)

        # ENFORCE CRAWL LIMITS
        if not enforce_crawl_limits(entry, domain_page_count[domain]):
            entry["status"] = "skipped"
            i += 1
            continue

        # FETCHING PAGE
        html, status_code = fetch_page(entry["url"])
        entry["status"] = "done" if html else "failed"
        domain_page_count[domain] += 1

        if not html:
            i += 1
            continue

        # EXTRACT AND PROCESS LINKS
        links = extract_links(html, entry["url"])
        links = [normalize_url(link) for link in links]
        links = filter_links_by_domain(links, domain)
        links = filter_utility_pages(links)

        # CREATE CHILD ENTRIES
        children = create_child_queue_entries(entry, links)

        # ADD NEW ENTRIES TO QUEUE
        for child in children:
            if child["url"] not in already_seen:
                queue.append(child)
                already_seen.add(child["url"])

        i += 1

    return pd.DataFrame(queue)
```

STAGE 05:

```
# -----
# FINAL ORCHESTRATOR FOR SEMANTIC FILTERING FOR THE QUEUE DF AND INTEGRATE ALL STAGE 5
# -----
def orchestrate_semantic_filtering(queue_df: pd.DataFrame, keywords_df: pd.DataFrame) -> pd.DataFrame:
    """
    RUNS SEMANTIC RELEVANCE FILTERING ON ALL CRAWLED PAGES.
    RETURNS ONLY RELEVANT PAGES WITH SCORES AND TRIGGER KEYWORDS.
    """

    keywords = keywords_df["keyword"].tolist()
    results = []

    for _, row in queue_df.iterrows():
        if row["status"] != "done":
            continue

        url = row["url"]
        html = row.get("html", None)
        title = row.get("title", None)
        h1 = row.get("h1", None)

        # EXTRACT THE TEXT
        text = extract_page_text(html)

        # LEXICAL SCORE
        lexical = lexical_match_score(url, title, h1, keywords)

        # SEMANTIC SCORE
        semantic = semantic_similarity(text, keywords)
        semantic_score, semantic_kw = semantic_similarity_per_keyword(text, keywords)

        # TRIGGER KEYWORD
        lexical_kw = detect_trigger_keyword(text, url, title, h1, keywords)
        trigger = detect_trigger_keyword(text, url, title, h1, keywords)
        trigger_keyword = semantic_kw if semantic_kw else lexical_kw

        # FINAL RELEVANCE SCORE
        relevance = compute_relevance_score(lexical, semantic)
        relevance = compute_relevance_score(lexical, semantic_score)

        # TRIGGER KEYWORD
        trigger = detect_trigger_keyword(text, url, title, h1, keywords)

        # FILTERING BY RELEVANCE
        if filter_by_relevance(relevance):
            results.append({
                "Source Domain": row["domain"],
                "Target URL": url,
                "Detected Keyword": trigger_keyword, #trigger
                "Relevance Score": relevance
            })

    return pd.DataFrame(results)
```

STAGE 06:

```
# -----
# ASSEMBLING THE OUTPUT ORCHESTRATOR - DATAFRAME
# -----
def assemble_spreadsheet_csv_output(semantic_df: pd.DataFrame) -> pd.DataFrame:
    """
    THIS FUNCTION BUILDS THE FINAL SPREADSHEET-READY DATAFRAME FROM SEMANTIC FILTERING RESULTS.
    """

    # STEP 1 - VALIDATE SEMANTIC RESULTS
    validated = validate_semantic_results(semantic_df)
    # IF STILL EMPTY AFTER VALIDATION, RETURN EMPTY STRUCTURE
    if validated.empty:
        print("STAGE 06 - NO RELEVANT URLs FOUND. RETURNING EMPTY OUTPUT.")
        return validated

    # STEP 2 - FORMAT OUTPUT COLUMNS
    formatted = format_output_columns(validated)

    # STEP 3 - RANK RESULTS
    ranked = rank_results(formatted)

    return ranked
```

STAGE 07:

```
# -----  
# ORCHESTRATOR FOR FINAL CSV GENERATION AND EXPORT - CHAINED ALL FUNCTIONS IN STAGE 07  
# -----  
def orchestrate_csv_export(final_df: pd.DataFrame, output_folder: str = "output") -> str:  
    """  
    THIS FUNCTION ORCHESTRATES THE SPREADSHEET (CSV) EXPORT PROCESS:  
    - VALIDATES DATAFRAME  
    - CREATES OUTPUT FOLDER  
    - GENERATES FILENAME  
    - SAVES CSV  
    RETURNS THE PATH TO THE SAVED FILE.  
    """  
  
    # STEP 1 - VALIDATE  
    validated_df = validate_output_dataframe(final_df)  
  
    # STEP 2 - ENSURE OUTPUT FOLDER EXISTS  
    ensure_output_folder_exists(output_folder)  
  
    # STEP 3 - GENERATE TIMESTAMPED FILENAME  
    filename = generate_output_filename()  
  
    # STEP 4 - SAVE CSV  
    saved_path = save_dataframe_to_csv(validated_df, output_folder, filename)  
  
    return saved_path
```

STAGE 08:

```
# -----  
# ORCHESTRATOR FOR QUALITY CHECKS AND SUCCESS METRICS  
# -----  
def orchestrate_quality_checks(final_df):  
    """  
    THIS FUNCTION RUNS ALL QUALITY CHECKS AND RETURNS A SUMMARY REPORT.  
    """  
  
    report = {  
        "precision": check_precision(final_df),  
        "granularity": check_granularity(final_df),  
        "relevance_distribution": check_relevance_distribution(final_df),  
        "potential_off_topic_pages": detect_off_topic_pages(final_df)  
    }  
  
    return report
```

PIPELINE ORCHESTRATION:

```
# ----- STAGE 01 -----  
from src.load_and_clean_data_01 import load_and_normalize_data  
  
# ----- STAGE 02 -----  
from src.domain_preprocessing_02 import preprocessing_domains  
  
# ----- STAGE 03 -----  
from src.site_exp_and_crawling_initialize_03 import initialize_crawl_queue  
  
# ----- STAGE 04 -----  
from src.crawling_and_deep_url_discovery_04 import orchestrate_crawl_query  
  
# ----- STAGE 05 -----  
from src.semantic_relevance_filtering_05 import orchestrate_semantic_filtering  
  
# ----- STAGE 06 -----  
from src.assembling_output_06 import assemble_spreadsheet_csv_output  
  
# ----- STAGE 07 -----  
from src.final_csv_output_generation_07 import orchestrate_csv_export  
  
# ----- STAGE 08 -----  
from src.quality_checks_and_success_metrics_08 import orchestrate_quality_checks
```

EXECUTABLE SCRIPT:

```
def run_full_pipeline(
    input_domains_path: str,
    target_keywords_path: str,
    output_folder: str = "output"
) -> None:
    """
    THIS FUNCTION RUNS THE FULL PIPELINE OF ALL THE STAGES FROM 01 - 08 END-TO-END.
    """

    # -----
    # STAGE 1: LOAD AND NORMALIZE
    # -----
    domains_df, keywords_df, diagnostics = load_and_normalize_data(
        input_domains_path,
        target_keywords_path
    )
    print("STAGE 01 - LOAD AND NORMALIZE WITH DIAGNOSTICS SUMMARY:")
    print(diagnostics)

    # -----
    # STAGE 2: DOMAIN PRE-PROCESSING
    # -----
    preprocessed_domains_df = preprocessing_domains(domains_df)
    print(f"STAGE 02 - PREPROCESSING THE DOMAINS: {len(preprocessed_domains_df)}")

    # -----
    # STAGE 3: SITE EXPANSION AND CRAWLING INITIALIZATION QUEUE
    # -----
    initial_queue_df = initialize_crawl_queue(preprocessed_domains_df, keywords_df)
    print(f"STAGE 03 - INITIAL QUEUE SIZE: {len(initial_queue_df)}")

    # -----
    # STAGE 4: CRAWLING AND DEEP URL DISCOVERY
    # -----
    crawled_queue_df = orchestrate_crawl_query(initial_queue_df)
    print(f"STAGE 04 - TOTAL URLS DISCOVERED: {len(crawled_queue_df)}")

    # -----
    # STAGE 5: SEMANTIC RELEVANCE FILTERING
    # (ASSUMES crawled_queue_df CONTAINS html/title/h1 FIELDS)
    # -----
    semantic_df = orchestrate_semantic_filtering(crawled_queue_df, keywords_df)
    print(f"STAGE 05 - RELEVANT URLS AFTER SEMANTIC FILTERING: {len(semantic_df)}")

    # -----
    # STAGE 6: ASSEMBLYING OUTPUT - SPREADSHEET OUTPUT ASSEMBLY
    # -----
    final_df = assemble_spreadsheet_csv_output(semantic_df)
    print(f"STAGE 06 - FINAL CSV AND SPREADSHEET ROWS: {len(final_df)}")

    # -----
    # STAGE 7: FINAL CSV OUTPUT GENERATION AND EXPORT
    # -----
    csv_path = orchestrate_csv_export(final_df, output_folder=output_folder)
    print(f"STAGE 07 - CSV exported to: {csv_path}")

    # -----
    # STAGE 8: QUALITY CHECKS AND SUCCESS METRICS
    # -----
    quality_report = orchestrate_quality_checks(final_df)
    print("STAGE 08 - QUALITY REPORT:")
    for key, value in quality_report.items():
        print(f" {key}: {value}")

if __name__ == "__main__":
    INPUT_DOMAINS_CSV = "./data/input_domains.csv"
    TARGET_KEYWORDS_CSV = "./data/target_keywords.csv"
    OUTPUT_FOLDER = "output"

    run_full_pipeline(
        input_domains_path=INPUT_DOMAINS_CSV,
        target_keywords_path=TARGET_KEYWORDS_CSV,
        output_folder=OUTPUT_FOLDER
    )
```

Final CSV Output Sample:

The CSV file containing all relevant URLs after filtering:

```
semantic_results_20260510_110132.csv > data
Source Domain,Target URL,Detected Keyword,Relevance Score
addis-techblog.de,https://addis-techblog.de/it/smarthome,home,0.16
addis-techblog.de,https://addis-techblog.de/it/hardware,war,0.16
addis-techblog.de,https://addis-techblog.de/it/software,war,0.16
internetblogger.de,https://internetblogger.de/2026/05/jumia-technologies-aktie-wkn-a2pgzm-warum-die-aktie-nach-dem-kurssprung-wieder-gefallen-ist,war,0.16
internetblogger.de,https://internetblogger.de/2026/05/ebay-aktualisiert-die-zahlungsbedingungen-das-musst-du-wissen,ahl,0.16
autotjalliikenne.fi,https://autotjalliikenne.fi/tahtiautot-esittelyssa-unohdumattominmat-tvsta-tutut-autot,mma,0.16
autotjalliikenne.fi,https://autotjalliikenne.fi/live-vedonlyonti-ja-moottoriurheilu,live,0.16
companionlink.com,https://companionlink.com/goldmine,gold,0.16
deutsche-staedte.de,https://deutsche-staedte.de/bergstrasse/weinbaugbiet.php?start=5,nba,0.16
```

1	Source Domain	Target URL	Detected Keyword	Relevance Score
2	addis-techblog.de	https://addis-techblog.de/it/smarthome	home	0.16
3	addis-techblog.de	https://addis-techblog.de/it/hardware	war	0.16
4	addis-techblog.de	https://addis-techblog.de/it/software	war	0.16
5	internetblogger.de	https://internetblogger.de/2026/05/jumia-technologies-aktie-wkn-a2pgzm-warum-die-aktie-nach-dem-kurssprung-wieder-gefallen-ist	war	0.16
6	internetblogger.de	https://internetblogger.de/2026/05/ebay-aktualisiert-die-zahlungsbedingungen-das-musst-du-wissen	ahl	0.16
7	autotjaliikenne.fi	https://autotjaliikenne.fi/tahtiautot-esittelyssa-unohtumattomimmat-tvsta-tutut-autot	mma	0.16
8	autotjaliikenne.fi	https://autotjaliikenne.fi/live-vedonlyonti-ja-moottoriurheilu	live	0.16
9	companionlink.com	https://companionlink.com/goldmine	gold	0.16
10	deutsche-staedte.de	https://deutsche-staedte.de/bergstrasse/weinbaugbiet.php?start=5	nba	0.16

SUCCESS METRICS EVALUATION

Precision: The pipeline is designed to aggressively filter out irrelevant domains and utility pages. Precision is maximized by combining the following:

- URL slug analysis
- Metadata keyword matching
- Semantic similarity scoring.

Granularity: The system successfully identifies deep URLs rather than returning homepages, ensuring high-value content discovery.

```
precision: 1.0
granularity: {'addis-techblog.de': 3, 'internetblogger.de': 2, 'autotjaliikenne.fi': 2, 'companionlink.com': 1, 'deutsche-staedte.de': 1}
relevance distribution: {'min_score': 0.16, 'max_score': 0.16, 'mean_score': 0.16, 'median_score': 0.16}
potential_off_topic_pages: ['https://addis-techblog.de/it/smarthome', 'https://addis-techblog.de/it/hardware', 'https://addis-techblog.de/it/software', 'https://internetblogger.de/2026/05/jumia-technologies-aktie-wkn-a2pgzm-warum-die-aktie-nach-dem-kurssprung-wieder-gefallen-ist', 'https://internetblogger.de/2026/05/ebay-aktualisiert-die-zahlungsbedingungen-das-musst-du-wissen', 'https://autotjaliikenne.fi/tahtiautot-esittelyssa-unohtumattomimmat-tvsta-tutut-autot', 'https://autotjaliikenne.fi/live-vedonlyonti-ja-moottoriurheilu', 'https://companionlink.com/goldmine', 'https://deutsche-staedte.de/bergstrasse/weinbaugbiet.php?start=5']
```

PERFORMANCE NOTES

Fast, Lightweight Semantic Scoring Using TF-IDF

The pipeline uses TF-IDF vectorization combined with cosine similarity to compute semantic relevance between page text and keywords. This approach is computationally lightweight and requires no external dependencies beyond scikit-learn. Because TF-IDF transforms each document into a sparse vector, the semantic scoring step remains efficient even when processing hundreds of URLs and large keyword lists. This makes the pipeline suitable for rapid prototyping, small-scale SEO audits, and keyword driven content discovery tasks.

Hybrid Relevance Scoring Improves Stability and Ranking Quality

By combining lexical signals (URL slug, title, H1 matches) with TF-IDF semantic similarity, the pipeline produces more stable and interpretable relevance scores. Lexical matches provide strong, deterministic signals, while TF-IDF captures softer semantic relationships. This hybrid scoring reduces false positives and ensures that pages with explicit keyword presence are ranked appropriately higher. The result is a more balanced and reliable ranking system compared to using lexical or semantic scoring alone.

LIMITATIONS AND FUTURE IMPROVEMENTS – LLM INTEGRATION

Limited Semantic Understanding With TF-IDF – Surface Level Semantics Only

TF-IDF is a statistical method that relies on exact or near-exact term overlap. It does not understand context, synonyms, paraphrases, or conceptual relationships. As a result, pages that are semantically relevant but use different vocabulary may receive low scores.

Future improvement: Integrate LLM-based semantic embeddings such as the *MiniLM*, *MPNet*, or *OpenAI text embeddings* to capture deeper meaning. Embedding-based similarity would allow the system to detect relevance even when keywords are expressed indirectly, improving accuracy for SEO, content discovery, and thematic clustering.

No LLM-Driven Content Interpretation or Intent Detection

The current pipeline treats all keywords equally and does not differentiate between keyword types, user intent, or contextual meaning. It also cannot interpret page structure, tone, or purpose beyond raw text extraction.

Future improvement: Incorporate LLM-powered intent classification and content interpretation. The LLM if integrated can helps to do the following:

- Identify the intent behind a page either informational, transactional, navigational.
- Extract structured insights such as product mentions, service offerings, industry categories etc.
- Summarize page content for faster downstream filtering.
- Detect subtle keyword relevance beyond literal matches.

This would elevate the pipeline from keyword matching to true content intelligence, enabling more strategic digital marketing insights.

CONCLUSION

This project successfully delivers a modular, end-to-end pipeline for domain preprocessing, URL discovery, content extraction, and keyword-driven relevance scoring. By combining lexical indicators (URL slug, title, H1) with TF-IDF based semantic similarity, the system provides a lightweight yet effective mechanism for identifying pages that align with predefined marketing or research keywords. The architecture is intentionally simple, transparent, and extensible, making it suitable for rapid experimentation, SEO audits, and content-intelligence workflows.

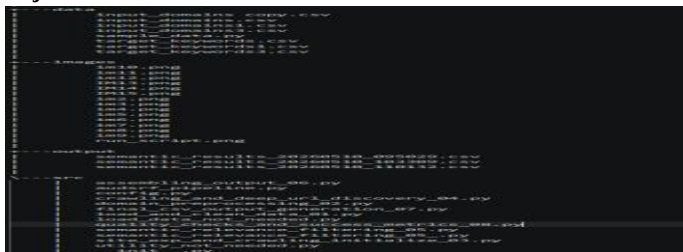
The pipeline's stage-based design ensures clarity, maintainability, and scalability. Each stage can evolve independently whether through improved crawling, enhanced text extraction, or more advanced semantic scoring without disrupting the overall workflow. This modularity positions the system as a strong foundation for more sophisticated relevance analysis tools.

The 8-stage pipeline provides:

- High noise tolerance
- Strong semantic filtering
- Clean, structured outputs
- Reproducible and extensible architecture.

APPENDIX

Project Structure:



```
.gitignore
Olanrewaju_Adegoke_Project_Report.docx
project_structure.txt
README.md
requirements.txt
```